LEARN BEING FORWARD

Programming — C Programming

Pointers & memory management

COMPLETE LEARNING GUIDE

What You'll Learn

Pointers are the single most defining feature of C — they expose memory addresses directly to the programmer, which is what makes C fast and close to the hardware, and also what makes it unforgiving: nearly every category of serious C bug (crashes, security vulnerabilities, memory corruption) traces back to a pointer pointing somewhere it shouldn't. Unlike garbage-collected languages, C requires the programmer to explicitly request memory from the heap with malloc/calloc/realloc and explicitly return it with free — there is no safety net if you forget, double-free, or keep using a pointer after the memory it refers to has been released. This sub-module builds pointer fundamentals from the address-of and dereference operators through pointer arithmetic, multi-level indirection, the dynamic memory allocation family, and the specific memory bugs that cause the most damage in real C programs.

1. Pointer Basics: & and *

Every variable lives at a memory address, and the & (address-of) operator retrieves that address as a pointer value. A pointer variable is declared with a type and an asterisk (int *p;), meaning 'p holds the address of an int.' The * operator, when applied to a pointer (not during declaration), dereferences it — it follows the address to read or write the value actually stored there. A pointer that doesn't yet point anywhere meaningful should be explicitly set to NULL, and dereferencing a NULL or otherwise invalid pointer is undefined behavior, typically crashing the program (a segmentation fault) rather than doing anything useful.

- int x = 5; int *p = &x; — p now holds x's address. *p reads/writes x's value through p.
- Always initialize pointers (to a valid address or NULL) — an uninitialized pointer holds a garbage address that may or may not crash when dereferenced.

```c
#include <stdio.h>

int main(void) {
    int x = 5;
    int *p = &x;

    printf("x = %d\n", x);    // 5
    printf("*p = %d\n", *p);  // 5 — dereferencing p reads x

    *p = 10; // writes through the pointer
    printf("x = %d\n", x);    // 10 — x itself changed
    return 0;
}
```

2. Pointer Arithmetic and Arrays

Incrementing a pointer moves it forward by exactly sizeof(the pointed-to type), not by one raw byte — p + 1 on an int* moves 4 bytes forward (on a typical platform), landing on the next int in memory, which is precisely why pointer arithmetic and array indexing are two notations for the same operation: arr[i] is defined to mean *(arr + i). This equivalence is why an array name used in an expression decays into a pointer to its first element, and why pointer arithmetic is only well-defined within (or one-past-the-end of) a single array — walking a pointer past that boundary and dereferencing it is undefined behavior even if it happens not to crash immediately.

- arr[i] and *(arr + i) are exactly equivalent — the compiler generates identical code for both.
- Pointer arithmetic is only valid within a single array (or one element past its end, for loop termination checks) — never across unrelated allocations.

```c
#include <stdio.h>

int main(void) {
    int arr[4] = {10, 20, 30, 40};
    int *p = arr; // array decays to a pointer to arr[0]

    for (int i = 0; i < 4; i++) {
        printf("%d ", *(p + i)); // identical to arr[i]
    }
    printf("\n"); // 10 20 30 40
    return 0;
}
```

3. Pointers to Pointers and Multi-Level Indirection

A pointer can itself be stored at an address, requiring a pointer-to-pointer (int **pp;) to hold that address — one extra level of * per extra level of indirection. This comes up whenever a function needs to modify the CALLER's pointer itself (not just the data it points to), since C's pass-by-value rules apply to pointers exactly as to any other type: a function receiving an int *p parameter gets a copy of that pointer, so reassigning p inside the function doesn't change the caller's pointer — only a int **pp parameter, dereferenced once to reach the real pointer, can do that. This pattern appears constantly in functions that allocate memory and need to hand the new pointer back to the caller.

```c
#include <stdio.h>
#include <stdlib.h>

void allocateAndSet(int **out, int value) {
    *out = malloc(sizeof(int)); // modifies the CALLER's pointer, via one level of indirection
    **out = value;
}

int main(void) {
    int *p = NULL;
    allocateAndSet(&p, 42); // pass the ADDRESS of p
    printf("%d\n", *p);     // 42
    free(p);
    return 0;
}
```

4. Dynamic Memory: malloc, calloc, realloc, free

malloc(size) requests size bytes from the heap and returns a void* to the start of that block, or NULL if the allocation failed — its contents are uninitialized garbage. calloc(count, size) allocates count*size bytes and additionally zero-initializes every byte, which is useful for arrays that should start at a known value. realloc(ptr, new_size) resizes a previously allocated block, possibly moving it to a new address and copying the existing contents — the returned pointer must always be used in place of the old one, since the old pointer may now be invalid even on success. Every successful malloc/calloc/realloc must eventually be paired with exactly one call to free(ptr) once the memory is no longer needed, or the program leaks that memory for its entire remaining lifetime.

- Always check malloc/calloc/realloc's return value for NULL before using it — allocation can and does fail, especially under memory pressure.
- ptr = realloc(ptr, newSize); is dangerous if realloc fails (returns NULL) — you lose the original pointer and leak it. Use a temporary variable instead.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *arr = malloc(n * sizeof(int));
    if (arr == NULL) {
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }

    for (int i = 0; i < n; i++) arr[i] = i * i;

    // Grow the array to hold 10 ints, preserving existing contents
    int *bigger = realloc(arr, 10 * sizeof(int));
    if (bigger == NULL) {
        free(arr); // original still valid — avoid losing it
        return 1;
    }
    arr = bigger;

    for (int i = 0; i < 10; i++) printf("%d ", arr[i]);
    printf("\n");

    free(arr); // release the memory exactly once
    return 0;
}
```

5. Common Memory Bugs: Leaks, Dangling Pointers, Double Free

A memory leak happens when allocated memory is never freed and the pointer to it is lost (overwritten or goes out of scope) — the program's memory usage grows until it's killed or the system runs out of memory. A dangling pointer is one that still holds the address of memory that has already been freed; the memory may be reused for something else, so reading or writing through a dangling pointer corrupts unrelated data unpredictably rather than failing cleanly. A double free occurs when free() is called twice on the same pointer, which corrupts the heap's internal bookkeeping and is a well-known source of security vulnerabilities. The standard discipline to avoid all three: free exactly once, and immediately set the pointer to NULL afterward so any accidental later use fails loudly (dereferencing NULL crashes immediately and visibly) instead of silently.

- After free(p);, immediately write p = NULL; — a second accidental free(p) or dereference then fails safely instead of corrupting memory.
- Tools like Valgrind (Linux/macOS) and AddressSanitizer (-fsanitize=address with gcc/clang) catch leaks, double-frees, and use-after-free automatically — use them routinely, not just when debugging a crash.

6. const with Pointers

const can qualify either the pointed-to data, the pointer itself, or both, and the position of const in the declaration determines which: const int *p means 'p points to data that can't be changed through p' (but p itself can be reassigned to point elsewhere); int *const p means 'p itself can't be reassigned, but the data it points to can be modified'; const int *const p locks both. Using const int *parameter for function parameters that only read data (never write through the pointer) documents intent clearly and lets the compiler catch accidental writes.

```c
#include <stdio.h>

void printArray(const int *arr, int size) { // promises not to modify arr's contents
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
        // arr[i] = 0; // would be a COMPILE ERROR — arr points to const data
    }
    printf("\n");
}

int main(void) {
    int nums[3] = {1, 2, 3};
    printArray(nums, 3);
    return 0;
}
```

7. Pointers as Function Arguments: Simulating Pass-by-Reference

Since C always passes arguments by value, the only way for a function to modify a caller's variable directly is to receive a pointer to it and dereference that pointer to write the new value — this is exactly how a correct swap() function must be written, and the same principle extends to any function that needs to produce more than one 'output' value (one via the return statement, the rest via pointer parameters that it writes through).

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void) {
    int x = 1, y = 2;
    swap(&x, &y); // pass addresses, not values
    printf("x=%d y=%d\n", x, y); // x=2 y=1
    return 0;
}
```

Common Pitfalls

- Dereferencing a pointer that was never initialized or was set to NULL, crashing with a segmentation fault.
- Forgetting to free() every successful malloc/calloc/realloc, leaking memory for the remaining life of the program.
- Using a pointer after the memory it refers to has been freed (a dangling pointer / use-after-free bug).
- Calling free() twice on the same pointer, corrupting the heap's internal bookkeeping.
- Assigning realloc's return value directly back to the original pointer variable without checking for NULL first, losing the original allocation on failure.
- Walking pointer arithmetic past the bounds of the array it points into, reading or writing unrelated memory.
- Not checking malloc/calloc's return value for NULL before dereferencing the result.

Key Takeaways

- & retrieves an address; * dereferences a pointer to read or write the value it points to.
- arr[i] and *(arr + i) are exactly equivalent — array indexing is pointer arithmetic in different notation.
- Every successful malloc/calloc/realloc must be paired with exactly one free(); set the pointer to NULL immediately after freeing it.
- A dangling pointer (use-after-free) and a double free are both undefined behavior and common sources of security vulnerabilities.
- A pointer-to-pointer is required when a function must modify the caller's own pointer variable, not just the data it points to.
- const int *p protects the pointed-to data from modification through p; int *const p protects p itself from reassignment.

Learn More

cppreference — Pointer declaration
https://en.cppreference.com/w/c/language/pointer

GeeksforGeeks — Dynamic Memory Allocation in C
https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/